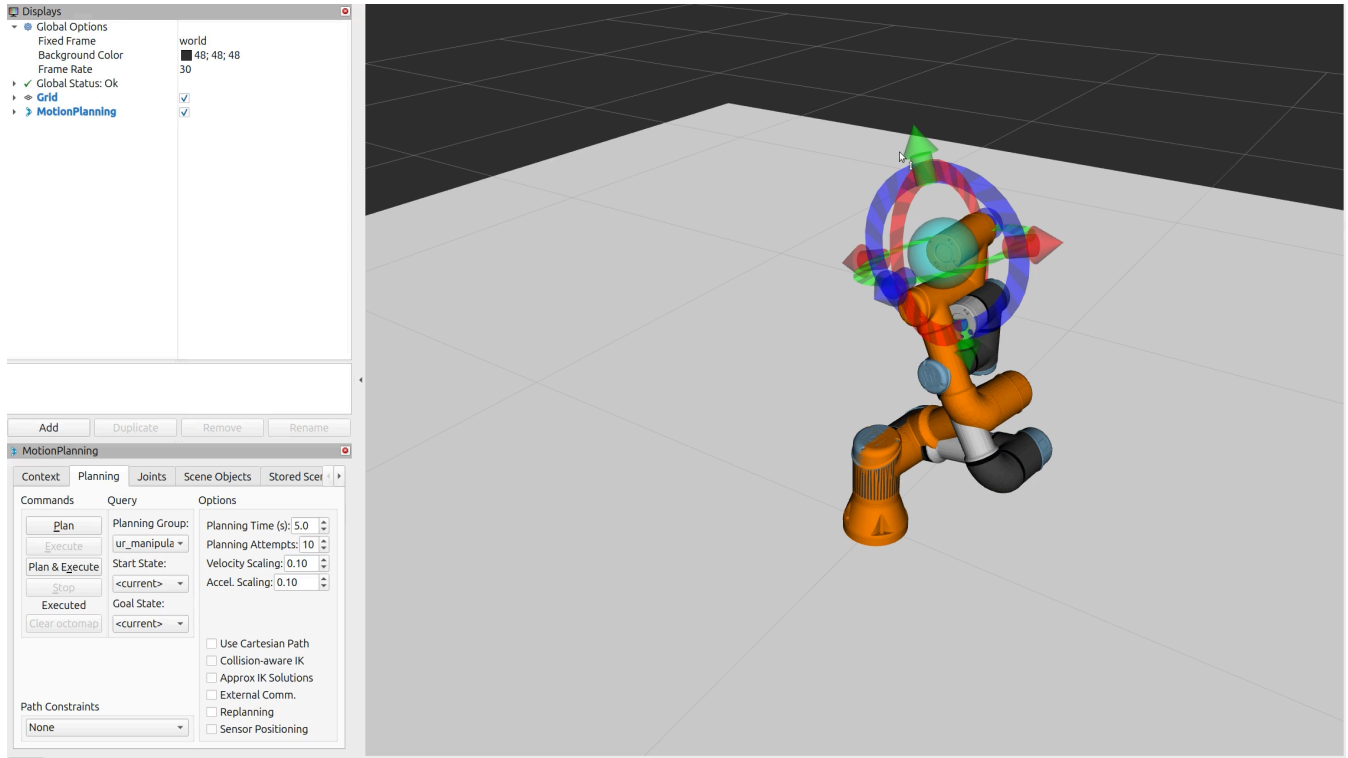
Kinematics

Autonomous Systems and
Mobile Robots · Week 6



UR3e arm: MoveIt plan and execute — ASMR Tutorial Week 6

Janosch Bajorath
j.bajorath@uni-muenster.de

Universität Münster

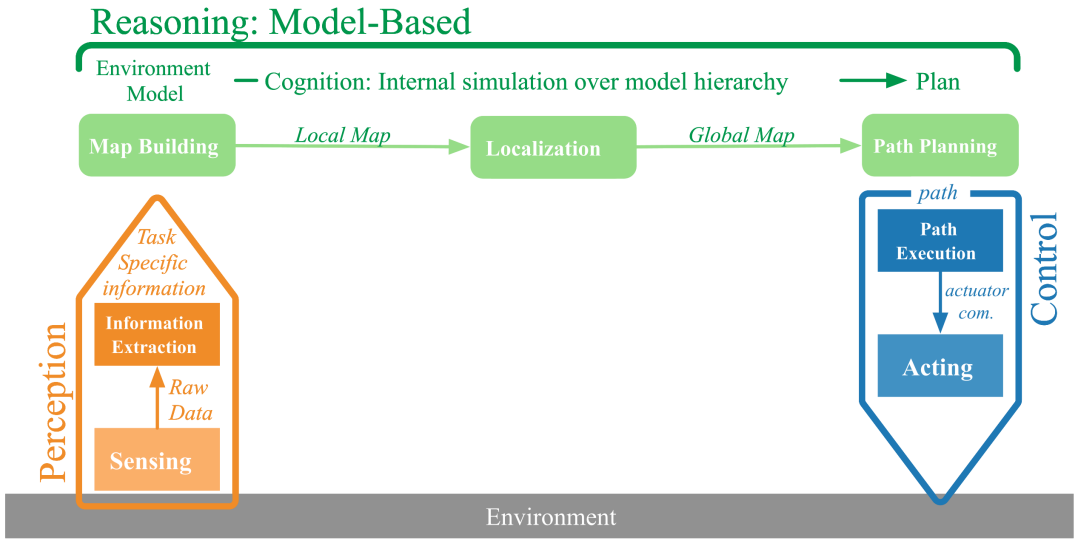
Recap

Where you are, where we go today

Semester at a Glance

#	Date	Theme
0	17.4	Python Intro + PRA Loop
1	24.4	ROS2 Basics + PID Control
—	*1.5*	*Public holiday — no session*
2	8.5	Robot Body and Blind Locomotion
3	15.5	Exteroception — LiDAR
4	22.5	MiniLab 1 Briefing
—	*29.5*	*Pfingsten — no session*
5	5.6	Troubleshooting — MiniLab 1
6	**12.6**	**FK/IK Robotic Manipulator** ← you are here
7	19.6	MiniLab 2 Briefing
8	26.6	Troubleshooting — MiniLab 2
9	3.7	Recap

Perceive · Reason · Act



Perceive

`/joint_states` — read the arm's current joint angles from encoders.

Reason

FK: joint angles → end-effector pose.

IK: target pose → joint angles to command.

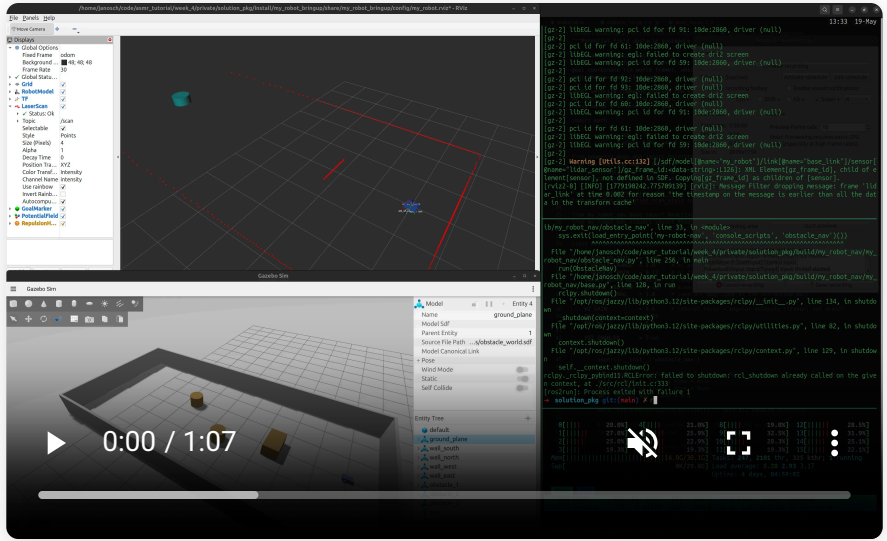
Act

Joint trajectory goal → controller moves the arm.

Last Week

Week 4 — MiniLab 1 Briefing

- Closed the full PRA loop on a wheeled robot
- **Reason**: reactive algorithm — LiDAR data → motion decision
- Layered rules: safety → avoidance → navigation
- ROS2 QoS: reliability and durability for `/scan` and latched topics



Today's Content

Theory

- Homogeneous transforms
- 2-link arm
- Forward kinematics
- Geometric IK
- Iterative IK
- Jacobian IK
- Singularities

MoveIt

- Scaling to 6-DOF
- The MoveIt pipeline

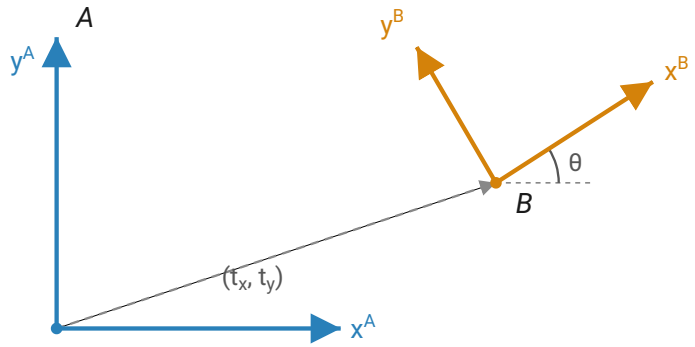
Exercises

- Notebook: `TwoLinkArm`
- ROS: UR3e + MoveIt exploration

Manipulator Kinematics

From joint angles to end-effector pose and back

Homogeneous Transformations



Frame B's origin sits at (t_x, t_y) in A; B's axes are rotated by θ from A's. (t is measured in A's axes.)

$$ {}^{A}_{B}T = \begin{bmatrix} \cos\theta & -\sin\theta & t_x \\ \sin\theta & \cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix} $$

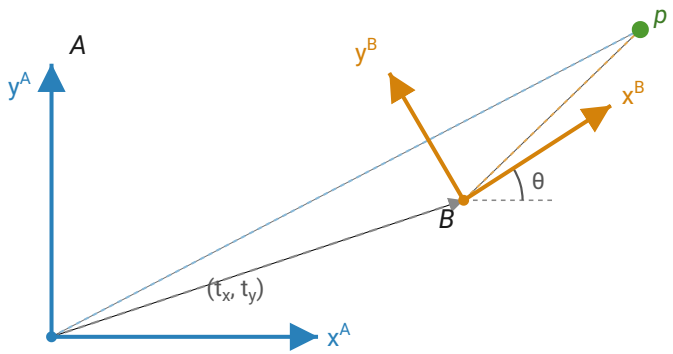
Superscript = reference frame · Subscript = child frame

$$ {}^{A}_{C}T = {}^{A}_{B}T \cdot {}^{B}_{C}T $$

Transforms chain by right-multiplication.

Same convention as ROS tf2: a transform is the **pose of the child in the parent**. Right-handed · X-fwd · Y-left · Z-up.

Transforming a Point



The same point p — seen from frame A (blue) and frame B (orange).

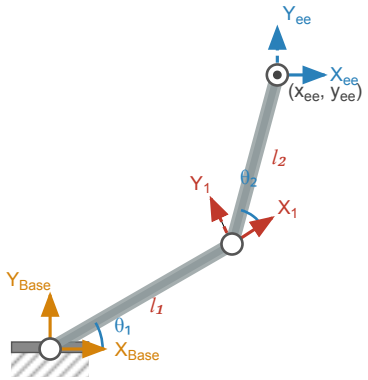
$$ {}^{A}\mathbf{p} = {}^{A}_{B}T \cdot {}^{B}\mathbf{p} $$

Frame B: rotated 90° CCW, origin at $(1, 0)$ in A.

$$ {}^{A}\mathbf{p} = \begin{bmatrix} 0 & -1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} $$

Point $(1, 0)$ in B is at $(1, 1)$ in A.

The 2-Link Arm



θ_1, θ_2	joint angles (positive = CCW)
l_1, l_2	link lengths
(x_{ee}, y_{ee})	end-effector position in Base frame

Think shoulder + elbow: θ_1 = shoulder, θ_2 = elbow, l_1/l_2 = upper arm / forearm, end-effector = fingertip.

Forward Kinematics

$$\overset{\text{Base}}{\text{ee}}T = \overset{\text{Base}}{1}T \cdot \overset{1}{\text{ee}}T$$

One transform per joint — rotate by θ_i, then step l_i along the **rotated** link axis → translation $(l_i \cos\theta_i,\ l_i \sin\theta_i)$. Frame 1 is at the elbow, Frame ee at the end-effector.

$$x_{\text{ee}} = l_1 \cos\theta_1 + l_2 \cos(\theta_1 + \theta_2)$$

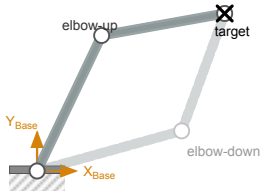
$$y_{\text{ee}} = l_1 \sin\theta_1 + l_2 \sin(\theta_1 + \theta_2)$$

$$\phi = \theta_1 + \theta_2$$

Position (x_{ee}, y_{ee}) is what you implement in `forward_kinematics()`; ϕ is the end-effector orientation in the base frame. **One configuration in → exactly one pose out.**

Geometric IK

Two solutions for every reachable target



elbow-up ($\theta_2 > 0$) · *elbow-down* ($\theta_2 < 0$)

Reaching for a glass: elbow above the table or below — same hand position, two postures.

Step 1 — find θ_2 (law of cosines):

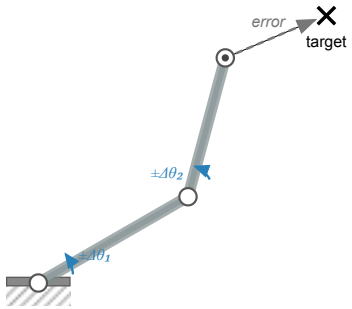
$$\theta_2^{\uparrow,\downarrow} = \pm \arccos\left(\frac{x^2 + y^2 - l_1^2 - l_2^2}{2\, l_1 l_2}\right)$$

Step 2 — each θ_2 gives its own θ_1:

$$\theta_1(\theta_2) = \operatorname{atan2}(y, x) - \operatorname{atan2}(l_2 \sin\theta_2,\ l_1 + l_2 \cos\theta_2)$$

Two solutions: $\left(\theta_1(\theta_2^{\uparrow}),\ \theta_2^{\uparrow}\right)$ and $\left(\theta_1(\theta_2^{\downarrow}),\ \theta_2^{\downarrow}\right)$. None if $|\cos\theta_2| > 1$ (out of reach).

Iterative IK

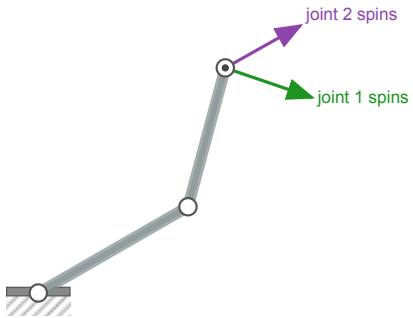


Each $\Delta\theta$ nudges the end-effector. Keep the nudges that shrink the error.

What if there's no closed form? (3+ links, 6-DOF)

1. Measure the **error**: current end-effector → target.
2. Nudge each joint a little ($\pm\Delta\theta$); keep what **shrinks** the error.
3. Repeat — the end-effector creeps to the target.

Jacobian IK



The two arrows at the end-effector — how it moves when each joint spins — **are the columns of J**.

The elegant form of "nudge & check" — the exact derivative of FK.

$$\dot{\mathbf{x}} = J(\mathbf{q})\,\dot{\mathbf{q}}$$

$$J = \begin{bmatrix} \partial x/\partial\theta_1 & \partial x/\partial\theta_2 \\ \partial y/\partial\theta_1 & \partial y/\partial\theta_2 \end{bmatrix}$$

Rows = output (x, y) · Columns = joints (θ_1, θ_2)

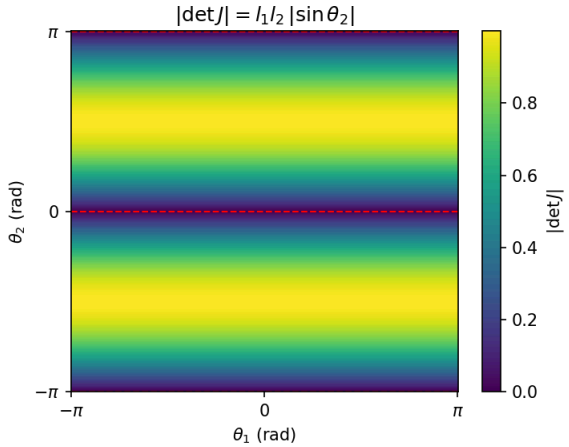
e.g. $\partial x/\partial\theta_1 = -l_1 \sin\theta_1 - l_2 \sin(\theta_1 + \theta_2)$

$$\dot{\mathbf{q}} = J^{+}\,\dot{\mathbf{x}}, \quad J^{+} = J^{-1}\ (\text{square})$$

Over-actuated arms (more joints than task dims): $J^{+} = (J^{\top}J)^{-1}J^{\top}$ ·

(breaks down when $\det J = 0$)

Singularity



$\det J = l_1 l_2 \sin\theta_2$ — zero whenever $\theta_2 = 0$ or $\pm\pi$, for any θ_1.

$$\det J = 0 \implies J \text{ not invertible}$$

The two velocity arrows go **parallel** — the arm loses a direction it can move. Small Cartesian motions then demand huge joint speeds.

- **Boundary** — arm fully **stretched** or **folded** ($\theta_2 = 0, \pm\pi$). The only kind a 2-link arm has.
- **Internal** — joint axes align mid-workspace. A **6-DOF** phenomenon; none on the 2-link.

MoveIt

The same math, scaled to 6-DOF

From 2 Links to 6-DOF

	2-link arm	UR3e (6-DOF)
Joints	2	6
IK solutions	2 (elbow-up / down)	Many configurations
Closed-form IK	Yes — law of cosines	Not in general*
Jacobian size	2 × 2	6 × 6

*General 6-DOF IK has no simple closed form — numerical solvers iterate the Jacobian update until convergence. (UR arms are a special case *with* known analytic IK, which is why MoveIt can use an analytic plugin.) MoveIt wraps this — and adds collision awareness.

MoveIt

1. **IK Solver** — given a target end-effector pose, numerically find joint angles $\mathbf{q}$ — the same iterate-the-Jacobian idea, scaled to 6 joints
2. **Planning Scene** — collision geometry: robot body + added objects (tables, boxes)
3. **Path Planner** — computes a collision-free joint-space trajectory (e.g. RRTConnect)
4. **Trajectory Execution** — sends the trajectory to the joint trajectory controller

> **Plan → Execute:** inspect the planned trajectory in RViz before committing the arm to move. The same goal can produce different paths on repeated calls — planners like RRTConnect are probabilistic.

Exercises

Notebook · ROS exploration

Overview

Notebook — `TwoLinkArm`

- Task 1: homogeneous transforms (`ht2d`)
- Task 2: forward kinematics
- Task 3: geometric IK (elbow-up / elbow-down)
- Task 4: Jacobian matrix
- Task 5: iterative Jacobian IK

ROS Exercise — UR3e (no coding)

- Task 1: node graph, `/joint_states` , TF tree
- Task 2: manual joint commands, `tf2_echo` verification
- Task 3: MoveIt — drag marker, Plan, Execute; collision object